

Compiled Intelligence

A Gradient-Free Path Toward Auditable and Self-Improving AI

Douglas Rawson
rawson.douglas@gmail.com

May 2026

Abstract

We present *Compiled Intelligence*—a paradigm for constructing and improving AI systems through deterministic weight manipulation rather than gradient-based optimization. Where current approaches treat neural network training as a black-box optimization problem, we treat model behavior as an engineering discipline: knowledge and capabilities are *compiled* directly into model weights via closed-form solutions, are fully auditable, and can be composed, versioned, and patched like software.

The core insight is that many capabilities currently “learned” through massive pretraining can instead be *baked in* through targeted activation engineering. We demonstrate this across multiple dimensions: exact multi-token behavioral control (solving the compound-probability collapse), persistent protocol-level steering, distributed knowledge storage at 50,000+ facts with zero cross-talk, and a systematic concept compilation pipeline.

We outline a staged roadmap from today’s compiled augmentation of pretrained models toward a regime where the compilation substrate itself becomes the primary intelligence fabric—reducing reliance on opaque pretraining in favor of auditable, composable capabilities. We discuss implications for AI safety, alignment, and the emerging field of mechanistic interpretability.

This paper is a technical vision document. It describes demonstrated capabilities and outlines research directions. It does not claim imminent AGI.

1 Introduction

The dominant paradigm in artificial intelligence is *emergent*: train large neural networks on vast corpora, and capabilities emerge from scale. This has produced remarkable systems, but at a cost: the resulting models are opaque, their knowledge is inextricably tangled across billions of parameters, and modifying their behavior requires full retraining—causing catastrophic forgetting of prior capabilities.

We propose an alternative: *compiled intelligence*. Instead of treating model behavior as something that must be learned through optimization, we treat it as something that can be engineered. Knowledge and capabilities are compiled directly into model weights through deterministic, closed-form operations. The compiled modules are auditable (every weight change has a known purpose), composable (modules can be stacked, versioned, and hot-swapped), and reversible (patches can be applied and removed without side effects).

This paper describes the compiled intelligence paradigm: the techniques that make it possible, the results achieved so far, the path toward greater capability, and the implications for AI safety and alignment.

2 The Compiled Intelligence Paradigm

2.1 Compilation vs. Training

Traditional neural network training solves:

$$\min_{\theta} \sum_{(x,y) \in \mathcal{D}} \mathcal{L}(f_{\theta}(x), y)$$

—an optimization problem over all parameters θ given a dataset $\mathcal{D}$. The solution is distributed, opaque, and fragile to modification.

Compiled intelligence takes a different approach. For a target behavior b , we compute:

$$\Delta\theta_b = \text{compile}(b, f_{\theta})$$

—a deterministic weight modification that induces behavior b without affecting other capabilities. The compilation function is closed-form, auditable, and bounded in scope.

2.2 Key Properties

1. **Deterministic:** Same inputs $\rightarrow$ same weight modification. No random seeds, no optimizer noise.
2. **Auditable:** Every injected vector, every expanded dimension, and every threshold has a documented purpose.
3. **Bounded:** Only targeted layers, positions, and decode steps are modified. The vast majority of parameters are untouched.
4. **Composable:** Multiple compiled modules can coexist through subspace allocation, trigger gating, and ordered plugin stacks.
5. **Reversible:** All modifications are applied via context managers with guaranteed rollback. The original model is always recoverable.
6. **Gradient-free:** No backward passes. No Adam. No SGD. The entire pipeline is closed-form linear algebra.

3 Core Techniques

3.1 The Residual Stream Model

A transformer decoder layer computes:

$$h_{\text{out}} = \text{FFN}(\text{LayerNorm}(h + \text{Attention}(h))) + (h + \text{Attention}(h))$$

The hidden state at the last sequence position encodes everything the model knows about the input. The `lm.head` projects this to vocabulary logits:

$$\text{logits} = W_{\text{lm}} \cdot \text{LayerNorm}(h[-1, :])$$

All token predictions flow through this single vector. Modifying it modifies what the model predicts next.

3.2 Unembedding Rows as Steering Vectors

The `lm_head` weight matrix $W_{\text{lm}} \in \mathbb{R}^{V \times d}$ has one row per vocabulary token. Adding a scaled copy of $W_{\text{lm}}[t, :]$ to the hidden state maximizes the logit for token t . This provides the mathematical foundation for using unembedding rows as steering vectors: they are the exact direction in hidden space that produces the desired token.

3.3 Per-Step Injection and the Compound-Probability Collapse

A static delta fails for multi-token sequences because the probability of correctly generating N tokens with a fixed bias decays exponentially:

$$P_{\text{correct}} = \prod_{k=1}^N P(t_k \mid \text{context}_{<k}, + \text{static delta}) < 1^N$$

Per-step injection replaces the compound product with N independent single-token problems by swapping the injected delta between forward passes. This reduces failure from exponential to linear in sequence length—the breakthrough that enabled reliable multi-token behavioral control.

3.4 Attention vs. FFN: Two Axes of Control

The attention mechanism and feed-forward network serve fundamentally different roles:

- **Attention injection** sets the model’s *protocol class*—the mode or style of response. Static deltas suffice because the protocol should remain constant throughout generation.
- **FFN injection** provides per-position *token precision*. Requires per-step delta swapping because different tokens are needed at different decode positions.

Combined, they provide complete behavioral control: attention sets the context, FFN drives the output.

3.5 Concept Compilation via Low-Rank Synthesis

For knowledge expressed as multiple (prompt, response) examples, we compute optimal FFN weight deltas via:

$$\Delta W = A^\dagger \cdot (B - A)$$

followed by SVD truncation to rank k . This produces (U, V) factor pairs per layer that, when applied as $y = \text{FFN}(x) + \alpha \cdot (x @ U^T) @ V$, induce the target behavior.

3.6 Compiled FFN Rules

For discrete knowledge (facts, rules, lookup tables), we expand FFN intermediate dimensions with trigger-gated pathways. Each rule is a rank-1 FFN addition:

$$\text{output} += \text{ReLU}(x \cdot \text{gate}_k - \theta_k) \cdot \text{value}_k$$

Empirically, this architecture holds 50,000+ rules in $d = 24$ dimensions with zero cross-talk and single-digit-second compilation times.

4 Key Results

4.1 Behavioral Control

- **Exact multi-token injection:** Per-step attention injection produces exact 10-token target phrases with zero effect on unrelated prompts. Verified on 0.5B, 1.5B, and 7B (4-bit) models.
- **Protocol-level shifts:** Positive/negative anchoring at layers 14–23 produces consistent JSON formatting across all prompt types, including previously resistant variations.
- **Mid-generation redirection:** The CodingEngine pattern detects trigger patterns in generated tokens, injects corrective text, and resumes generation seamlessly.
- **Composed control:** Attention (protocol) + FFN (tokens) simultaneous injection with alpha co-tuning provides both mode and content control.

4.2 Knowledge Compilation

- **50,000+ compiled FFN rules:** Zero cross-talk in $d=24$ dimensional embedding space. 150,000/150,000 correct predictions. Compilation time: 5.7 seconds on CPU.
- **Hot-add knowledge extension:** New rules appended as tensor rows without retraining. Existing rules preserved byte-for-byte.
- **Concept compilation pipeline:** 89 ConceptPack rules compiled from automotive diagnostic library with 1,494/1,494 strict-correct predictions across continuous knowledge growth checkpoints.

4.3 Compiled Reasoning Primitives

- **Two-layer induction head:** 100/100 random-pair induction, semantic generalization with synonym queries (dog/puppy, mom/mum).
- **Chained multi-step reasoning:** Two-hop transitive inference, 100% top-1 at $K=4$ facts. Composed from stacked induction primitives without named reasoning steps.
- **Compiled arithmetic:** 1000/1000 exact numeric addition via lookup-table carry circuit. Zero gradient steps.
- **Gated induction:** Abstention when no anchor signal exists. Perfect operating-point separation at $\tau = 0.9$ cosine threshold.

5 The Self-Improvement Loop

The compiled intelligence paradigm enables a *self-improvement loop* qualitatively different from gradient-based fine-tuning:

1. **Monitor:** The system continuously evaluates its own outputs against correctness criteria (syntax validators, test suites, factual consistency checks, human feedback).
2. **Detect:** Systematic failure patterns are identified—not single mistakes, but recurring classes of errors (e.g., “JSON outputs are missing closing braces,” “arithmetic on numbers $> 10^4$ fails”).
3. **Diagnose:** The model analyzes its own failure, identifies the root cause (wrong protocol? wrong token? missing fact?), and generates corrected examples.

4. **Compile:** A compilation module computes the optimal weight delta to fix the detected pattern, using the appropriate mechanism (per-step injection for exact outputs, anchoring for protocols, FFN rules for facts).
5. **Verify:** The compiled fix is tested against held-out examples and negative controls. Parity verification ensures zero drift on unrelated capabilities.
6. **Deploy:** The validated patch is applied—either as a persistent wrapper or baked directly into weights.

Critically, this loop can be *model-driven*. The language model itself participates in steps 2–3 (detection and diagnosis), while steps 4–5 are handled by deterministic compilation machinery that requires no gradient computation.

5.1 Live Self-Distillation

A particularly powerful instance of this loop: a strong model (e.g., 120B) can serve as a *teacher* for a smaller model (e.g., 0.5B). Residual stream activations are captured from the teacher during correct responses, and optimization targets for the student are computed as:

$$\Delta W_{\text{student}} = \text{pinv}(A_{\text{student}}) \cdot (A_{\text{teacher}} - A_{\text{student}})$$

This is gradient-free knowledge distillation—the student is rewired to match the teacher’s internal representations at specific layers, without any backward pass.

6 Path to Advanced Capability

We outline a staged roadmap from today’s compiled augmentation toward systems where compilation is the primary capability fabric.

6.1 Stage 1: Compiled Augmentation (Current)

The model is a standard pretrained transformer. Compiled modules are layered on top for specific capabilities: fact injection, behavioral formatting, safety filtering, domain expertise. The modules are plugin-based and reversible. This stage is **operational today**.

6.2 Stage 2: Heavy Compilation

As the module library grows, the distinction between “base model” and “compiled capabilities” blurs. The model carries hundreds or thousands of compiled modules covering knowledge, reasoning patterns, and behavioral protocols. New capabilities are added primarily through compilation rather than retraining. The codebase already demonstrates this at the scale of tens of thousands of rules.

6.3 Stage 3: Circuit Synthesis

Individual compiled mechanisms (induction heads, arithmetic circuits, fact lookup tables) are composed into *circuits*—multi-step, multi-layer computational pathways that implement complex capabilities (planning, code generation, scientific reasoning). These circuits are synthesized from specifications, not learned from data. The compiled induction and arithmetic work demonstrates the viability of this approach at small scale.

6.4 Stage 4: Minimal-Pretrain Bootstrap

A small base model (tens of millions of parameters) is pretrained on a modest corpus to establish basic linguistic competence. Compiled circuits are then layered on to provide reasoning, knowledge, and behavioral control. The compilation infrastructure carries most of the capability burden, reducing the need for massive pretraining. This inverts the current paradigm: instead of “train big, hope capabilities emerge,” we “compile capabilities, use pretraining only for the substrate.”

6.5 Stage 5: Recursive Self-Improvement

The compilation architecture itself is compiled. The detection→diagnosis→compilation→verification loop becomes a capability module that the system can apply to improve its own compilation machinery. This creates a virtuous cycle: better compilation → better self-diagnosis → better compilation. The compiled modules are versioned, auditable, and subject to formal verification.

7 Implications for Safety and Alignment

The compiled intelligence paradigm offers several structural advantages for AI safety:

1. **Decomposability:** Capabilities are modular and independently auditable. A safety researcher can inspect exactly which weight modifications correspond to which behaviors.
2. **Intervenability:** Problematic behaviors can be surgically removed (negative anchoring, suppressor rules) without affecting other capabilities.
3. **Bounded impact:** Each compiled module affects only the layers, positions, and triggers it targets. The blast radius of any modification is known and limited.
4. **Provenance:** Every weight change has a recorded purpose, source, and verification result. The audit trail is complete.
5. **Reversibility:** Any compiled module can be removed, restoring the model to its prior state. There is no “unlearning” problem.
6. **Deterministic verification:** Correctness can be verified through exact functional tests rather than statistical evaluation.

These properties do not solve alignment—they make it *tractable*. Compiled modules still require correct specifications, and malicious or incompetent compilation could produce harmful behavior. But the structural properties of the paradigm—auditability, boundedness, reversibility—mean that verification and correction are far more feasible than in opaque, fully-trained models.

8 Limitations and Open Problems

1. **Scale of compilation:** Current demonstrations involve thousands to tens of thousands of compiled rules. Scaling to millions of rules requires addressing the noise budget (already characterized in our scale-wall experiments).
2. **Trigger gating:** Current trigger mechanisms (substring matching) are brittle. Learned triggers (cosine routing, classifier-based gating) are under development but not yet production-grade.
3. **Cross-module interference:** While individual modules are verified in isolation, interactions between multiple simultaneously-active modules are not fully characterized.

4. **Dynamic behaviors:** The compilation paradigm excels at static knowledge and behavioral patterns. Dynamically adapting behaviors (e.g., real-time strategy changes in complex environments) are an open area.
5. **The bootstrap question:** Stage 4 (minimal-pretrain bootstrap) requires a base model with sufficient linguistic competence to serve as a compilation substrate. The minimum viable pretraining scale for this is unknown.
6. **Verification completeness:** While compiled modules can be tested against specific probes, proving that a module has no unintended side effects across all possible inputs remains an open problem.
7. **Scalability of self-improvement:** The recursive self-improvement loop (Stage 5) has not been demonstrated. Whether the compilation machinery can effectively improve itself is an empirical question.

9 Related Work

The compiled intelligence paradigm draws from and contributes to several research traditions:

- **Mechanistic interpretability:** Work on induction heads (Olsson et al.), transformer circuits (Elhage et al.), and activation engineering (Turner et al.) provides the conceptual framework for understanding which weight modifications produce which behavioral effects.
- **Model editing:** Techniques like ROME, MEMIT, and MEND edit specific facts in model weights. Compiled intelligence generalizes this to arbitrary behavioral modifications.
- **LoRA and parameter-efficient fine-tuning:** Low-rank adaptation trains small weight matrices appended to existing layers. Our approach synthesizes these matrices deterministically rather than through gradient descent.
- **Neuro-symbolic AI:** The combination of symbolic knowledge structures (ConceptPacks) with neural substrates (transformer FFNs) echoes neuro-symbolic approaches, but with direct weight compilation rather than separate reasoning modules.
- **AI alignment:** Interpretability-based assurance, scalable oversight, and control techniques all benefit from the auditability and intervenability properties of compiled modules.

10 Call for Collaboration

The compiled intelligence paradigm is in its early stages. We invite collaboration from researchers in:

- **AI safety and alignment:** Red-teaming compiled modules, developing formal verification approaches, studying cross-module interaction effects.
- **Mechanistic interpretability:** Characterizing how compiled modules interact with existing model circuits, developing tools for visualizing and debugging compiled pathways.
- **Systems and infrastructure:** Building scalable compilation pipelines, optimizing for large module libraries, developing deployment infrastructure for hot-swappable capabilities.
- **Cognitive science and neuroscience:** The compilation paradigm—modular, composable, versioned capabilities—bears structural resemblance to theories of modular cognition. Cross-disciplinary insights are welcome.

Interested researchers are encouraged to open issues or discussions on the project repository at <https://github.com/DRawson5570/compiled-injection>, or contact the author directly.

11 Conclusion

Compiled intelligence offers a different path forward: one where AI capabilities are engineered rather than emerged, where knowledge is auditable rather than opaque, and where improvement is deterministic rather than stochastic. It does not replace gradient-based learning—pretraining on large corpora remains the most effective way to establish linguistic and conceptual foundations. But it provides a *complementary* paradigm for building on those foundations with precision, auditability, and control.

The techniques described here are real, demonstrated, and published with OpenTimestamp and Zenodo provenance. The roadmap is speculative but grounded. The invitation to collaborate is genuine.

This paper establishes prior work on the Compiled Intelligence paradigm as of May 2026.

Repository: <https://github.com/DRawson5570/compiled-injection>

Contact: rawson.douglas@gmail.com